

CS 341 Week 4 Lab — Mini-memcheck Worksheet

Warm-up

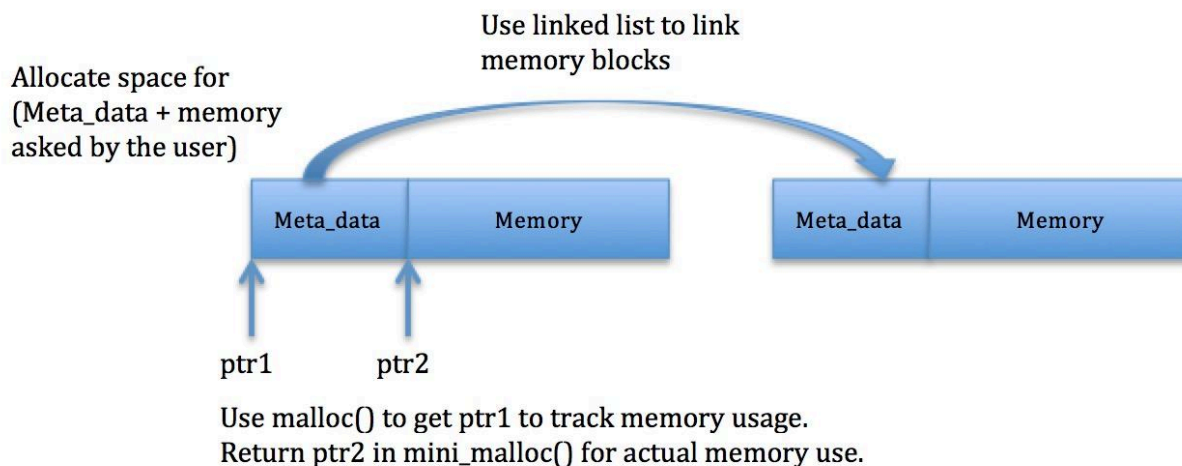
Suppose an `int *p` lives at address `0x0FE4`.

Address	Contents
<code>0x0FE0</code>	<code>42</code>
<code>0x0FE4</code>	<code>0x0FE0</code>
<code>0x0FE8</code>	<code>7</code>

- `p = _____` `*p = _____` `&p = _____`
- Someone changes the row at `0x0FE4` to hold `0x0FE8` instead.
Now `*p = _____`, and `p = _____`
- Starting from the original table, someone changes the row at `0x0FE0` to hold `99`.
 - Does `p` change? **yes / no**
 - Does `*p` change? **yes / no**

The metadata block

For each allocation, `mini_malloc` asks the real `malloc` for enough memory to hold a **metadata** struct followed by the **payload**, the bytes the user asked for. It then returns the address of the payload, not the address the real `malloc` gave it. The user never sees the metadata sitting in front of their data.



Mini-memcheck keeps the metadata for every allocation in a **linked list**, so that when the program exits it can walk the list and report anything never freed.

```
C/C++
typedef struct metadata {
    struct metadata *next;      /* next allocated block */
    size_t size;               /* bytes the user requested */
} metadata;
```

On a 32-bit machine, pointers and `size_t` are both 4 bytes.

Suppose the real malloc returned `metadata *md = 0x2000`.

1. Fill in the table

Expression	Address (hex)	Bytes past md
md		
md + 1		
md + sizeof(metadata)		

2. Which one do you hand to the user? _____
3. `md + sizeof(metadata)` is wrong by a factor of _____.
Explain in one sentence why it overshoots:

Walking the list

This table shows part of the **heap**. Each row is 4 bytes. Mini-memcheck's blocks were each returned by a separate call to the real malloc. Rows marked *(not ours)* do not belong to any block in this list. Touching them is a bug, even though the program usually won't crash.

The first block starts at **head** = 0x1000

Address	Memory	Address	Memory
0x1000	0x1020	0x102C	...
0x1004	16	0x1030	...
0x1008	...	0x1034	(not ours)
0x100C	...	0x1038	(not ours)
0x1010	...	0x103C	(not ours)
0x1014	...	0x1040	0x0
0x1018	(not ours)	0x1044	8
0x101C	(not ours)	0x1048	...
0x1020	0x1040	0x104C	...
0x1024	12	0x1050	(not ours)
0x1028	...		

Block A — starts at head

1. Block A's metadata starts at _____. Shade every row of the table that holds block A's metadata.
2. **next** = _____ **size** = _____
3. Block A's payload starts at _____. Given **metadata** ***md** pointing at block A's metadata, write an expression using **md** that gives the payload address:

4. What is the address of the last row that belongs to block A? _____
5. Draw a box around every row belonging to block A. Label it A.

The rest of the list

Keep following next until the list ends. Fill in one column per block you find, box each one on the table, and label them B, C, ... in order.

Blocks	
metadata starts at	
next	
size	
payload starts at	
last row	

1. How many blocks are in the list? _____
How did you know you'd reached the last one?

2. You just followed next from head to the end of the list. Write that as code:

```
C/C++
metadata *p = _____;
while ( _____ ) {
    /* ... inspect this block ... */
    p = _____;
}
```

3. When the program exits, mini-memcheck walks this list to build the leak report. If the program exited right now, how many bytes would be reported as leaked?

Freeing a block

Step 1: find the metadata

The user calls `free()` on block B's payload, starting at `0x1028`.

C/C++

```
void mini_free(void *payload) {  
    metadata *md = /* ??? */ ;  
    ...  
}
```

1. Write your own expression in the first row. Then work out where every row lands, including yours.

	metadata *md =	Lands at	Correct?
yours			
Student A	(metadata *)(payload - 1);		
Student B	payload - sizeof(metadata);		
Student C	(metadata *)payload - sizeof(metadata);		

2. B lands on the right address. Why shouldn't you write it? *Hint: what is sizeof(void)?*

Step 2: make sure it's really a block

3. A buggy program calls `free()` on `0x1018` from the above table.
Your expression gives _____. What is actually stored there? Why does `mini_free` have to search the list before trusting `md`?

Step 3: unlink it

4. Update the table to remove block B from the list. Change as few rows as possible: cross out the old value and write the new one.
5. The row you changed belongs to block _____. `mini_free` started with only `md`, pointing at block B. Why can't it get to that row from block B, and how does it find the row instead?

- Suppose the user had freed block A instead. What would change?

Step 4: give the memory back

- For block B, `mini_malloc` asked the real `malloc` for _____ bytes. `mini_free` calls the real `free` on address _____. `total_memory_freed` goes up by _____. Why isn't that the number from real `malloc`?

Placement Strategy

This section isn't part of this week's Mini Memcheck assignment. You'll use it in the Malloc System Project later in the semester.

An allocator manages a 64-byte heap that starts empty. It ran the code below, but you aren't told which placement strategy it uses. To keep the numbers simple, **ignore metadata and alignment**: each block takes exactly the number of bytes requested.

C/C++

```
1 a = malloc(8);      4 d = malloc(20);    7 free(b);
2 b = malloc(12);     5 e = malloc(8);      8 free(d);
3 c = malloc(8);      6 f = malloc(8);      9 free(f);
```

Reverse engineering: which strategy?

- Draw the heap after lines 1–6, placing each block with first fit. Your last block should end exactly at 64. Then mark the blocks freed in lines 7–9:



Now `g = malloc(8)` and `h = malloc(10)` run. This is the heap afterwards:

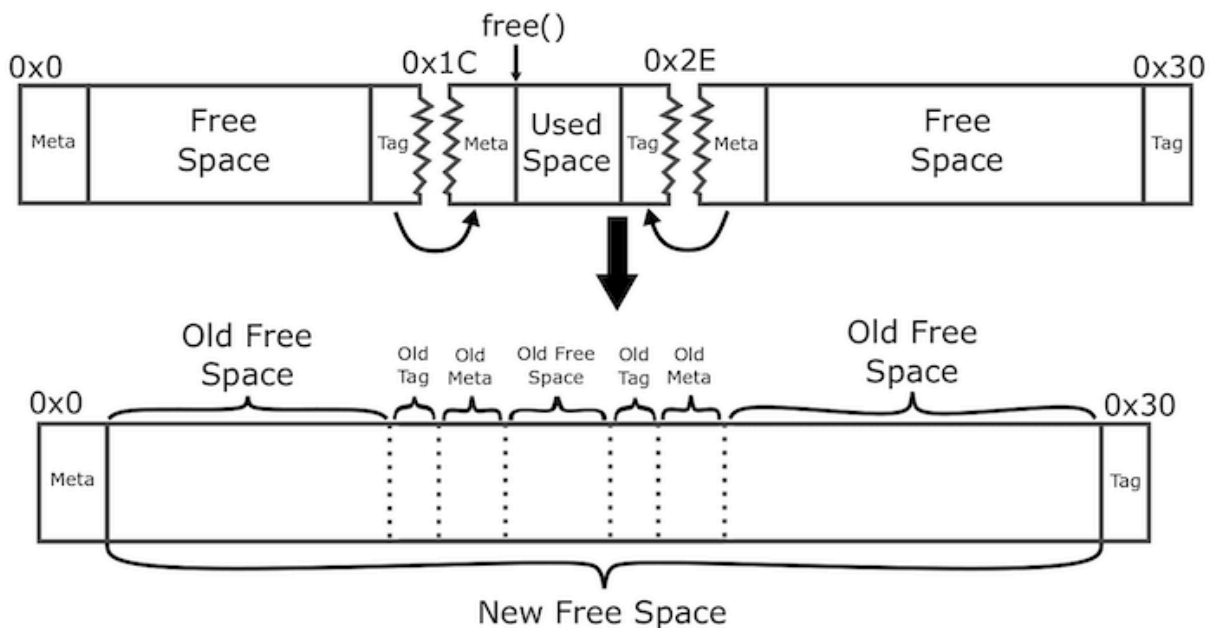
	0	8	18	20	28		48	56	64
block	a	h		c	free			e	g
size	8	10	2	8	20			8	8

- Using your holes from Q1, work out where each strategy would have placed g:
 First fit → _____ Best fit → _____ Worst fit → _____
- Of the three strategies, which one matches the picture? The allocator that placed g and h uses _____.
- With that same strategy, `h = malloc(10)` is placed at _____, leaving _____ bytes of that hole unused.
 Is that internal or external fragmentation? _____

Block Coalescing

This section isn't part of this week's Mini Memcheck assignment. You'll use it in the Malloc System Project later in the semester.

Block coalescing merges a freed block with any free neighbors next to it, turning several small free blocks into one larger one.



Every block here has an 8-byte metadata (Meta) at the front, and a 4-byte boundary tag (BTag) at the back.

Three allocated blocks, A, B and C, sit right next to each other and fill an 80-byte heap:

Meta	Space A	Tag	Meta	Space B	Tag	Meta	Space C	Tag
8B	16B	4B	8B	12B	4B	8B	16B	4B

1. Add the three Space regions only: _____ bytes are usable by programs right now.

The program calls **free(A)**, then **free(C)**, then **free(B)**.

2. After **free(A)** and **free(C)**, do any of them merge? **yes / no**
Why?

3. Now **free(B)** runs with coalescing. Draw the result, labelling the Meta and BTag that survive:

4. Compared with Q1, coalescing gained _____ extra usable bytes, without freeing any new block.

CS 341 Week 4 Lab — Worksheet Answer Keys

Warm-up

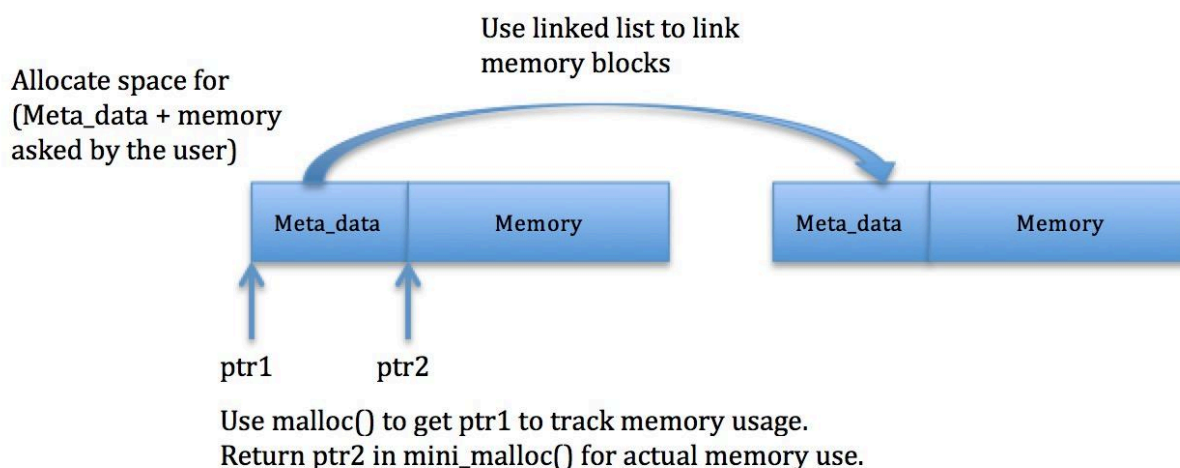
Suppose an `int *p` lives at address `0x0FE4`.

Address	Contents
<code>0x0FE0</code>	<code>42</code>
<code>0x0FE4</code>	<code>0x0FE0</code>
<code>0x0FE8</code>	<code>7</code>

- `p = 0x0FE0` `*p = 42` `&p = 0x0FE4`
- Someone changes the row at `0x0FE4` to hold `0x0FE8` instead.
Now `*p = 7` , and `p = 0x0FE8`
- Starting from the original table, someone changes the row at `0x0FE0` to hold `99`.
 - Does `p` change? **no**
 - Does `*p` change? **yes, becomes 99**

The metadata block

For each allocation, `mini_malloc` asks the real `malloc` for enough memory to hold a **metadata** struct followed by the **payload**, the bytes the user asked for. It then returns the address of the payload, not the address the real `malloc` gave it. The user never sees the metadata sitting in front of their data.



Mini-memcheck keeps the metadata for every allocation in a **linked list**, so that when the program exits it can walk the list and report anything never freed.

```
C/C++
typedef struct metadata {
    struct metadata *next;      /* next allocated block */
    size_t size;               /* bytes the user requested */
} metadata;
```

On a 32-bit machine, pointers and `size_t` are both 4 bytes.

Suppose the real malloc returned `metadata *md = 0x2000`.

1. Fill in the table

Expression	Address (hex)	Bytes past md
<code>md</code>	<code>0x2000</code>	<code>0</code>
<code>md + 1</code>	<code>0x2008</code>	<code>8</code>
<code>md + sizeof(metadata)</code>	<code>0x2040</code>	<code>64</code>

2. Which one do you hand to the user? `md + 1`
3. `md + sizeof(metadata)` is wrong by a factor of `8`.
Explain in one sentence why it overshoots:

`md` is a `metadata *`, so adding `n` moves `n` whole structs. `sizeof(metadata)` is `8`, so it moves $8 \times 8 = 64$ bytes. "It scales by the size twice" is also accepted.

Walking the list

This table shows part of the **heap**. Each row is 4 bytes. Mini-memcheck's blocks were each returned by a separate call to the real malloc. Rows marked *(not ours)* do not belong to any block in this list. Touching them is a bug, even though the program usually won't crash.

The first block starts at `head = 0x1000`

Address	Memory	Address	Memory
0x1000	0x1020 0x1040	0x102C	...
0x1004	16	0x1030	...
0x1008	...	0x1034	(not ours)
0x100C	...	0x1038	(not ours)
0x1010	...	0x103C	(not ours)
0x1014	...	0x1040	0x0
0x1018	(not ours)	0x1044	8
0x101C	(not ours)	0x1048	...
0x1020	0x1040	0x104C	...
0x1024	12	0x1050	(not ours)
0x1028	...		

Block A — starts at head

- Block A's metadata starts at **0x1000**. Shade every row of the table that holds block A's metadata.
- `next = 0x1020` `size = 16`
- Block A's payload starts at **0x1008**. Given `metadata *md` pointing at block A's metadata, write an expression using `md` that gives the payload address: **`md + 1`**
Also correct: `(char *)md + sizeof(metadata)`
- What is the address of the last row that belongs to block A? **0x1014**
- Draw a box around every row belonging to block A. Label it A.

The rest of the list

Keep following next until the list ends. Fill in one column per block you find, box each one on the table, and label them B, C, ... in order.

Blocks	B	C
metadata starts at	0x1020	0x1040
next	0x1040	0x0
size	12	8
payload starts at	0x1028	0x1048
last row	0x1030	0x104C

1. How many blocks are in the list? **3**
How did you know you'd reached the last one?

C's next is 0x0 (NULL), so it is a tail block in the list.

2. You just followed next from head to the end of the list. Write that as code:

```
C/C++
metadata *p = head;
while (p != NULL) {
    /* ... inspect this block ... */
    p = p->next;
}
```

Also correct: `while (p), while (p != 0)`

3. When the program exits, mini-memcheck walks this list to build the leak report. If the program exited right now, how many bytes would be reported as leaked?

36 (= 16 + 12 + 8)

Freeing a block

Step 1: find the metadata

The user calls `free()` on block B's payload, starting at `0x1028`.

C/C++

```
void mini_free(void *payload) {  
    metadata *md = /* ??? */ ;  
    ...  
}
```

1. Write your own expression in the first row. Then work out where every row lands, including yours.

	metadata *md =	Lands at	Correct?
yours	<code>(metadata *) payload - 1;</code>	<code>0x1020</code>	yes
Student A	<code>(metadata *) (payload - 1);</code>	<code>0x1027</code>	no
Student B	<code>payload - sizeof(metadata);</code>	<code>0x1020</code>	see Q2
Student C	<code>(metadata *) payload - sizeof(metadata);</code>	<code>0x0FE8</code>	no

Also correct: `(metadata *) ((char *) payload - sizeof(metadata))`

2. B lands on the right address. Why shouldn't you write it? *Hint: what is `sizeof(void)`?*

payload is a `void *`. `sizeof(void)` is not defined in standard C, so arithmetic on a `void *` is not allowed. gcc accepts it as an extension and treats it like `char *`, which is why the address comes out right.

Step 2: make sure it's really a block

3. A buggy program calls `free()` on `0x1018` from the above table. Your expression gives `0x1010`. What is actually stored there? Why does `mini_free` have to search the list before trusting `md`?

Block A's payload (user data, not metadata).

Why: Pointer arithmetic can't tell whether real metadata is there. To check, `mini_free` compares `md` against every node in the list, since those are the only real metadata addresses.

Step 3: unlink it

4. Update the table to remove block B from the list. Change as few rows as possible: cross out the old value and write the new one.

- The row you changed belongs to block **A**. `mini_free` started with only `md`, pointing at block B. Why can't it get to that row from block B, and how does it find the row instead?

The list is singly linked: B's metadata points forward to C, and nothing in B points back to A. `mini_free` walks from head, remembering the previous block, until it reaches B.

- Suppose the user had freed block A instead. What would change?

head changes from `0x1000` to `0x1020`. No row in the table changes.

Step 4: give the memory back

- For block B, `mini_malloc` asked the real `malloc` for **20** bytes. `mini_free` calls the real `free` on address `0x1020`. `total_memory_freed` goes up by **12**. Why isn't that the number from real `malloc`?

`md->size` counts only the bytes the user requested, not including the metadata

Placement Strategy

This section isn't part of this week's Mini Memcheck assignment. You'll use it in the Malloc System Project later in the semester.

An allocator manages a 64-byte heap that starts empty. It ran the code below, but you aren't told which placement strategy it uses. To keep the numbers simple, **ignore metadata and alignment**: each block takes exactly the number of bytes requested.

C/C++

```
1 a = malloc(8);      4 d = malloc(20);    7 free(b);
2 b = malloc(12);     5 e = malloc(8);    8 free(d);
3 c = malloc(8);      6 f = malloc(8);    9 free(f);
```

Reverse engineering: which strategy?

- Draw the heap after lines 1–6, placing each block with first fit. Your last block should end exactly at 64. Then mark the blocks freed in lines 7–9:

	0	8	20	28	48	56	64
block	a	b	c	d	e	f	
size	8	12	8	20	8	8	

Now `g = malloc(8)` and `h = malloc(10)` run. This is the heap afterwards:

	0	8	18	20	28		48	56	64
block	a	h		c	free			e	g
size	8	10	2	8	20			8	8

2. Using your holes from Q1, work out where each strategy would have placed `g`:

First fit → **8** Best fit → **56** Worst fit → **28**

3. Of the three strategies, which one matches the picture? The allocator that placed `g` and `h` uses **best fit**.

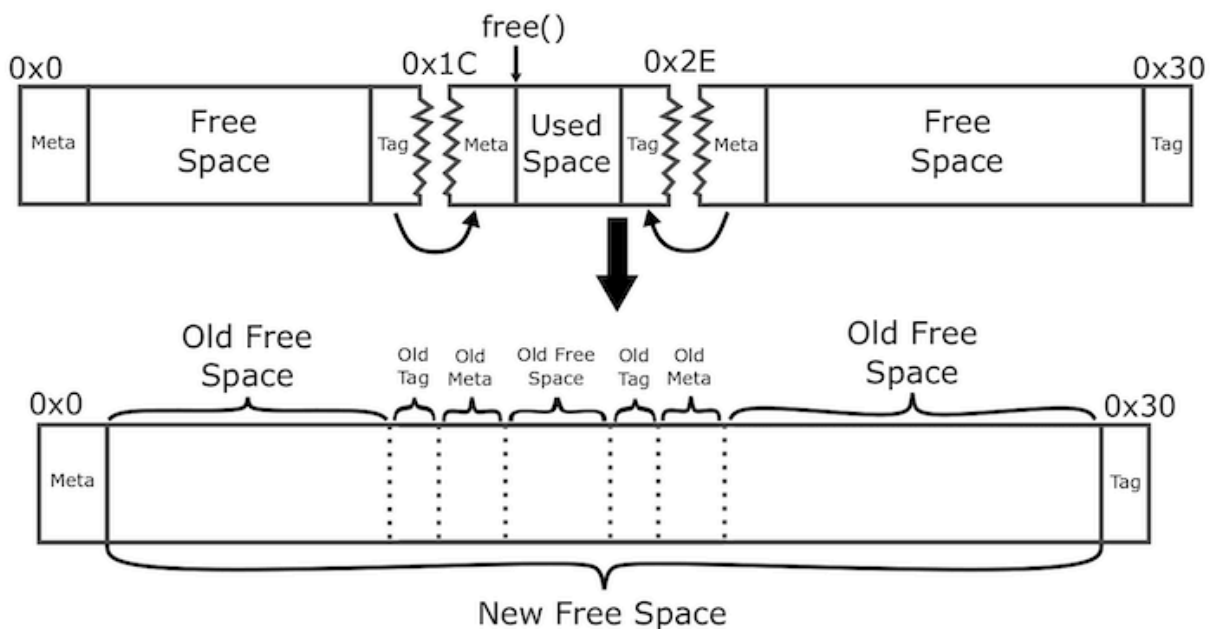
4. With that same strategy, `h = malloc(10)` is placed at **8**, leaving **2** bytes of that hole unused.

Is that internal or external fragmentation? **External fragmentation**

Block Coalescing

This section isn't part of this week's Mini Memcheck assignment. You'll use it in the Malloc System Project later in the semester.

Block coalescing merges a freed block with any free neighbors next to it, turning several small free blocks into one larger one.



Every block here has an 8-byte metadata (Meta) at the front, and a 4-byte boundary tag (BTag) at the back.

Three allocated blocks, A, B and C, sit right next to each other and fill an 80-byte heap:

Meta	Space A	Tag	Meta	Space B	Tag	Meta	Space C	Tag
8B	16B	4B	8B	12B	4B	8B	16B	4B

1. Add the three Space regions only: $16 + 12 + 16 = 44$ bytes are usable by programs right now.

The program calls `free(A)`, then `free(C)`, then `free(B)`.

2. After `free(A)` and `free(C)`, do any of them merge? **no**
Why?

A and C are not neighbors; block B, still allocated, sits between them.

3. Now `free(B)` runs with coalescing. Draw the result, labelling the Meta and BTag that survive:

Meta	Space	Tag
8B	68B	4B

4. Compared with Q1, coalescing gained **24** extra usable bytes, without freeing any new block.